Hacettepe University — Computer Engineering
CMP 720 EMBEDDED SYSTEMS DESIGN

Spring 2026 — Mini In-Class Project Series Part 1 - SAMPLE SOLUTION

Lab 1: System Design & Requirements Specification

SafePulse — Embedded Patient Monitoring System

Week 2: Specifications, Modeling & Requirements

Duration:	1 – 1.5 hours
Tools Required:	Pen/pencil, this handout (printed or digital)
Deliverable:	Completed template

1 Context & Overview

In this first hands-on session, you will design an embedded patient vital signs monitoring system called **SafePulse**, before writing a single line of code. Over the coming weeks, you will progressively implement, test, and validate this system using the Renode simulation framework on an STM32F4 platform.

SafePulse is a safety-critical cyber-physical system that continuously monitors a patient's heart rate, blood oxygen saturation (SpO2), and body temperature. When vital signs breach clinically dangerous thresholds, the system must trigger an alarm within a hard real-time deadline. Failure to do so could result in patient harm or death.

Why design first? In medical device development (governed by standards such as IEC 62304), implementation without a clear specification is not just bad practice, it is a regulatory violation. A well-defined specification serves as the contract that your implementation must satisfy and your tests must verify. Today, you write that contract.

2 Learning Objectives

By the end of this session, you should be able to:

1. Decompose a system-level concept into an embedded hardware/software block diagram identifying sensors, processing elements, communication interfaces, and actuators.

2. Define measurable functional requirements (what the system does) and non-functional requirements (how well it does it), with particular attention to timing constraints.

3. Model system behavior as a finite state machine with clearly defined states, transitions, and guard conditions.

4. Reason about the safety implications of requirement violations in a critical embedded system.

3 System Description

You are designing an embedded monitoring station for a hospital bedside. The system has the following characteristics:

3.1 Sensors (Inputs)

Sensor	Interface	Data	Notes
Heart Rate / SpO2	I2C	BPM, % saturation	Combined pulse oximeter. Normal SpO2: 95–100%, critical: below 90%
Body Temperature	I2C	°C (fixed-point)	Normal: 36.1–37.2°C, fever: above 38.0°C, critical: above 40.0°C

3.2 Processing Platform

STM32F4 microcontroller (ARM Cortex-M4) simulated in Renode. Key peripherals available: I2C controller, hardware timers (TIM2–TIM5), UART for serial console output, and GPIO for alarm indicators. In particular, general-purpose timers such as TIM2–TIM5 can be configured to generate a periodic "tick" interrupt (e.g., every 200 ms) that drives sensor sampling. A common pattern is: (1) the timer interrupt service routine (ISR) sets a lightweight `sample_due` flag (or posts an event), and (2) the main loop (or a high-priority task) performs the I2C reads and updates the state machine. This avoids busy-wait delays, keeps sampling jitter low, and helps ensure the 100 ms alarm deadline is met.

3.3 Outputs / Actuators

Output	Interface	Purpose
Alarm Indicator	GPIO	LED/buzzer activated when patient vitals are critical. Must activate within the hard deadline.
Nurse Station Display	UART	Serial console output simulating a remote display. Periodically logs current readings and system status.

3.4 Critical Timing Constraint

HARD REAL-TIME REQUIREMENT: When SpO2 drops below 90%, the alarm output must be activated within **100 milliseconds** of the sensor reading that detected the breach. Missing this deadline in a real system could result in patient death due to undetected hypoxemia.

4 Part 1: System Block Diagram

Draw a system block diagram. Your diagram should include:

a) All sensor blocks with their communication interfaces (I2C bus connections) labeled

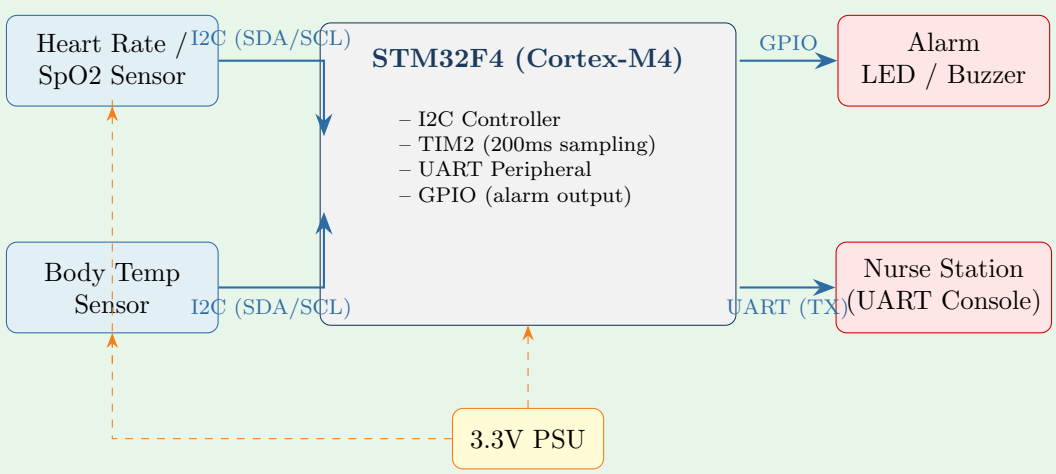
b) The STM32F4 MCU block showing relevant internal peripherals (I2C controller, timers, UART, GPIO). Refer to STM32F4 datasheet if necessary.

c) All output blocks (alarm indicator, nurse station display) with their interfaces

d) Data flow arrows showing the direction of information (sensor → MCU → outputs)

e) Power supply connections (even if simplified as a single block)

[Draw your system block diagram]



Key features of a good solution:

- Both sensors share a single I2C bus (SDA/SCL lines) with different device addresses.
- MCU internals show which peripherals map to which function: I2C for sensors, TIM for

periodic sampling, UART for display, GPIO for alarm.

- Data flow is clearly left-to-right: sensors → MCU → outputs.
- Power supply shown even if simplified.

5 Part 2: Requirements Specification Table

Complete the requirements tables below. Example requirements (FR-01, FR-02, NFR-01, NFR-02) are provided. You must add at least **4 additional functional requirements** and **2 additional non-functional requirements**. Consider: sensor sampling, data processing, alarm behavior, communication/logging, failure handling, and timing.

5.1 Functional Requirements

ID	Description	Priority	Verification
FR-01	The system shall read heart rate (BPM), SpO2 (%), and body temperature (°C) from the I2C sensor at a fixed sampling interval of 200 ms.	Must Have	Test / Timing
FR-02	The system shall activate the alarm GPIO output within 100 ms when SpO2 drops below 90%.	Must Have	Test / Timing
FR-03	*[Your requirement here]*	*[Priority]*	*[Method]*
FR-04	*[Your requirement here]*	*[Priority]*	*[Method]*
FR-05	*[Your requirement here]*	*[Priority]*	*[Method]*
FR-06	*[Your requirement here]*	*[Priority]*	*[Method]*
FR-07	*[Your requirement here]*	*[Priority]*	*[Method]*
FR-08	*[Your requirement here]*	*[Priority]*	*[Method]*

Example additional functional requirements:

FR-03	The system shall log current heart rate, SpO2, temperature, and system state via UART at a rate of 1 Hz.	Must Have	Test
FR-04	The system shall deactivate the alarm when SpO2 returns to $\geq$ 94% for at least 3 consecutive readings (hysteresis to prevent alarm flapping).	Must Have	Test
FR-05	The system shall enter a WARNING state and increase the logging rate to 2 Hz when SpO2 falls between 90–94% or temperature exceeds 38.0°C.	Should Have	Test
FR-06	The system shall detect I2C sensor communication failure (no ACK within 3 consecutive attempts) and enter a SENSOR_FAULT state.	Must Have	Test
FR-07	The system shall display a startup self-test banner on UART confirming all peripherals are initialized before entering the NORMAL state.	Should Have	Inspection
FR-08	When in CRITICAL or SENSOR_FAULT state, the alarm GPIO shall output a toggling pattern (500 ms on/off) rather than steady-on, to distinguish from a stuck GPIO.	Could Have	Test

5.2 Non-Functional Requirements

ID	Description	Category	Metric
NFR-01	Alarm activation latency from threshold breach detection to GPIO assertion shall not exceed 100 ms under any system load.	Timing / Safety	$\leq$ 100 ms
NFR-02	The system shall detect an I2C sensor communication failure (no ACK) within 500 ms and report it via UART.	Reliability	$\leq$ 500 ms
NFR-03	*[Your requirement here]*	*[Category]*	*[Metric]*
NFR-04	*[Your requirement here]*	*[Category]*	*[Metric]*
NFR-05	*[Your requirement here]*	*[Category]*	*[Metric]*

Example additional non-functional requirements:

NFR-03	Sensor sampling jitter shall not exceed ± 10 ms from the 200 ms target interval.	Timing	≤ 10 ms jitter
NFR-04	The system shall operate continuously for at least 72 hours without requiring a restart or exhibiting memory leaks.	Reliability	≥ 72 h uptime
NFR-05	UART log messages shall follow a consistent, machine-parseable format (e.g., CSV) to enable automated post-processing.	Maintainability	Parseable format

6 Part 3: System Behavior — Finite State Machine

Design a finite state machine (FSM) that models the patient monitoring behavior of SafePulse. Your FSM should capture how the system transitions between different alert levels based on sensor readings.

6.1 3a. State Definitions

Complete the following state table. A skeleton with suggested states is provided — you may modify, add, or remove states as you see fit.

State	Description	Alarm Output	Display
NORMAL	All vital signs within healthy ranges. Routine monitoring active.	OFF	Logging at 1 Hz
WARNING	*[Define: what conditions?]*	*[Alarm?]*	*[Display?]*
CRITICAL	*[Define: what conditions?]*	*[Alarm?]*	*[Display?]*
SENSOR_FAULT	*[Define: how detected?]*	*[Alarm?]*	*[Display?]*

WARNING	One or more vitals in concerning range: SpO2 90–94%, or temp 38.0–40.0°C, or HR outside 50–120 BPM.	OFF (or slow blink)	Logging at 2 Hz, "WARNING" prefix
CRITICAL	Any vital at dangerous level: SpO2 $<$ 90%, or temp $>$ 40.0°C, or HR $<$ 40 or $>$ 150 BPM.	ON (toggling 500 ms)	Logging at 2 Hz, "CRITICAL ALERT" on each line
SENSOR_FAULT	I2C read fails for 3 consecutive attempts (≈600 ms). Cannot trust readings.	ON (steady – distinct from CRITICAL)	"SENSOR FAULT" message, last known readings displayed

6.2 3b. State Transitions

Define the transitions between states. For each transition, specify the guard condition (what triggers it) and any action that occurs during the transition.

From	To	Guard Condition	Transition Action
NORMAL	WARNING	*[e.g., SpO2 < 95%]*	*[Actions?]*
WARNING	CRITICAL	*[Define condition]*	*[Actions?]*
CRITICAL	WARNING	*[Define recovery]*	*[Actions?]*
WARNING	NORMAL	*[Define recovery]*	*[Actions?]*
ANY	SENSOR_FAULT	*[Define fault detection]*	*[Actions?]*
SENSOR_FAULT	NORMAL	*[Define recovery]*	*[Actions?]*
[From?]	*[To?]*	*[Add more as needed]*	*[Actions?]*

NORMAL	WARNING	SpO2 $\in$ [90, 94%] OR temp > 38.0°C OR HR outside [50, 120]	Log "ENTERING WARNING"; increase log rate to 2 Hz
WARNING	CRITICAL	SpO2 < 90% OR temp > 40.0°C OR HR < 40 or > 150	Activate alarm GPIO within 100 ms; log "CRITICAL ALERT"
CRITICAL	WARNING	All critical thresholds cleared for $\geq$3 consecutive readings	Deactivate alarm; log "DOWNGRADE TO WARNING"
WARNING	NORMAL	All vitals in normal range for $\geq$5 consecutive readings	Reduce log rate to 1 Hz; log "RETURNING TO NORMAL"
ANY	SENSOR_FAULT	3 consecutive I2C NACK errors	If was in CRITICAL: keep alarm active; log "SENSOR FAULT — ALARM MAINTAINED"; otherwise activate alarm (fail-safe)
SENSOR_FAULT	NORMAL	Successful I2C read AND readings are in normal range	Deactivate alarm; log "SENSOR RECOVERED"
NORMAL	CRITICAL	SpO2 < 90% (direct jump, skip WARNING)	Activate alarm GPIO immediately; log "CRITICAL ALERT"

Design notes:

- **Hysteresis:** Requiring consecutive normal readings before downgrading prevents alarm

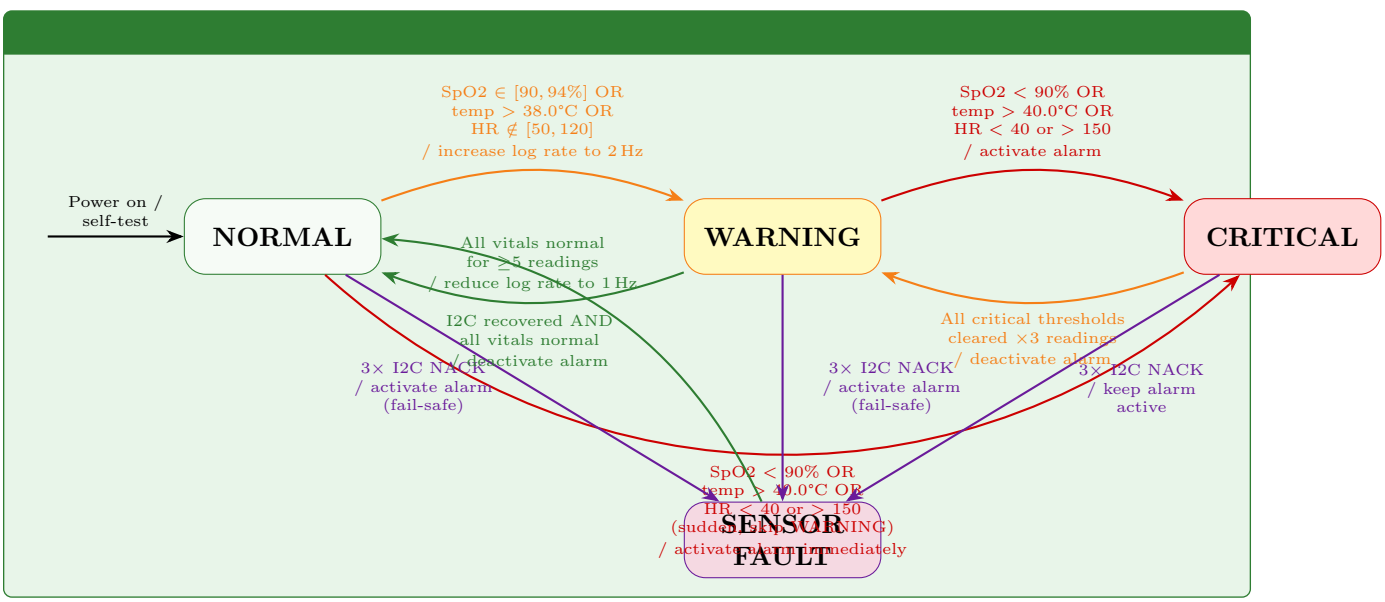
flapping from noisy sensor data.

- **Direct NORMAL→CRITICAL:** If SpO2 drops suddenly below 90%, we skip WARNING entirely — safety takes priority over orderly state progression.
- **Fail-safe SENSOR_FAULT:** When sensor fails, we assume the worst and activate the alarm rather than silently disabling monitoring.

6.3 3c. State Machine Diagram

Draw a state machine diagram representing the states and transitions defined above. Use standard notation: rounded rectangles for states, arrows for transitions labeled with `[guard] / action`.

[Draw your state machine diagram here — hint: start with an initial arrow pointing to NORMAL]



7 4. Design Reflection Questions

Answer the following briefly.

Q1: What happens if the sensor sampling rate (FR-01) is not met consistently? How does this affect the alarm deadline (NFR-01)? Describe a concrete scenario.

[Your answer]

If sampling is delayed (e.g., due to a blocking UART print inside the ISR), the system may read the sensor at 350 ms instead of 200 ms. If SpO2 dropped below 90% at $t = 100$ ms but the next read doesn't happen until $t = 350$ ms, the breach is detected 250 ms late. Adding the 100 ms alarm processing deadline, the patient could wait 350 ms before the alarm fires — 3.5× the allowed deadline. In that window, hypoxemia could cause brain damage. This demonstrates that sampling latency directly compounds alarm latency; NFR-01 is impossible to meet if FR-01 is violated.

Q2: Should the system transition directly from CRITICAL back to NORMAL if readings suddenly return to healthy ranges, or should it require sustained healthy readings for a period? Justify your design choice and its safety implications.

[Your answer]

The system should **not** transition directly from CRITICAL to NORMAL. Requiring sustained healthy readings (e.g., 3–5 consecutive readings ≈ 0.6–1.0 s) provides hysteresis that prevents alarm flapping from noisy or intermittent sensor data. A single healthy reading could be a sensor glitch; if the system silences the alarm and the patient is actually still in distress, the consequences are life-threatening. The trade-off is a brief period of unnecessary alarming during genuine recovery, which is vastly preferable to a missed alarm during genuine distress. This is the "fail-safe" design principle: when uncertain, default to the safer action (keep alarming).

Q3: If a sensor fault occurs while the patient is already in a WARNING or CRITICAL state, what should the system do? Is it safer to maintain the alarm or to enter a separate SENSOR_FAULT state? What are the trade-offs?

[Your answer]

The system should enter SENSOR_FAULT but **maintain the alarm if it was previously active**. The reasoning: if the patient was already in CRITICAL and then the sensor fails, silencing the alarm because "we don't have data" could be fatal. The fail-safe approach is to assume the patient condition persists (or worsened, possibly causing the fault) until proven otherwise. The trade-off: if the sensor simply disconnected and the patient is fine, the alarm continues unnecessarily — but this is acceptable because a false alarm is far less costly than a missed alarm in a medical context. The SENSOR_FAULT state should also use a distinct alarm pattern (e.g., steady vs. toggling) so staff can distinguish "patient critical" from "equipment malfunction."

8 Looking Ahead

Keep your completed template safe — you will use it as the specification for the remainder of the SafePulse project:

Week 3: You will set up the STM32F4 simulation in Renode and map your block diagram to actual simulated peripherals.

Week 4: You will implement the I2C sensor driver and timer-based sampling, and verify your FR-01 and NFR-01 timing requirements.

Week 5: You will port the system to Zephyr RTOS with multiple tasks and priorities — your state machine design will become real code.

Week 6: You will write automated tests that validate your implementation against the requirements table you completed today.

Key takeaway: Every line of code you write in weeks 3–6 should be traceable back to a requirement in your table. Every test you write in week 6 should verify a requirement from today. This is how professional embedded systems are built.